

LAB Assignment No 12

Case Study: IoT-Based Smart Hospital Management System using Computer Vision and IoT

The healthcare industry is rapidly adopting IoT and Artificial Intelligence technologies to improve patient care, hospital monitoring, automation, and emergency response systems. This case study focuses on developing a Smart Hospital Management System using IoT sensors, ESP32 controllers, Computer Vision, Cloud Platforms, and Web Dashboards.

The proposed system can monitor patients, detect medical emergencies, identify authorized persons using face recognition, and provide real-time hospital data access remotely through cloud technologies.

Problem Statement:

Traditional hospital monitoring systems face several issues:

- Manual patient monitoring
- Delayed emergency response
- Unauthorized access to ICU rooms
- Lack of real-time monitoring
- Difficulty in remote patient observation
- No centralized smart dashboard system

The proposed IoT-based smart hospital system solves these problems using automation and real-time monitoring.

Objectives of the Case Study

1. To develop a smart hospital monitoring system using IoT.
2. To integrate Computer Vision for face detection/recognition.
3. To monitor patient health parameters in real-time.
4. To send hospital data to cloud platforms.
5. To control and monitor devices remotely from anywhere.
6. To develop a web dashboard for visualization.
7. To improve hospital security and automation.

Proposed System Overview

The system combines:

- IoT Sensors
- ESP32 Microcontroller
- Computer Vision using Python OpenCV
- Firebase/ThingSpeak Cloud
- Web Dashboard
- Remote Monitoring

The system performs:

- Patient temperature monitoring
- Heartbeat monitoring
- Oxygen level monitoring
- ICU access control using face recognition
- Emergency alert generation
- Real-time data visualization
- Remote monitoring through internet

Working Principle

Step-by-Step Working

1. Sensors collect patient data.
2. ESP32 reads sensor values.
3. Camera captures live video.
4. OpenCV detects faces/persons.
5. Data is sent to cloud platforms.
6. Dashboard displays real-time values.
7. Alerts generated in emergency conditions.
8. Doctors can monitor system remotely.

Features of the Proposed System

- Face Detection for authorized staff
- Smart ICU Monitoring
- Patient Vital Monitoring
- Real-Time Alerts
- Cloud Data Storage
- Web Dashboard Monitoring
- Mobile Access
- Remote Device Control
- Automated Room Monitoring

Hardware Components

Component	Purpose
ESP32	Main IoT Controller
Webcam/USB Camera	Face Detection
Pulse Sensor	Heart Rate Monitoring
MAX30100/MAX30102	Oxygen & Heartbeat Sensor
DHT11/DHT22	Temperature & Humidity
LM35	Body Temperature

PIR Sensor	Motion Detection
MQ Gas Sensor	Oxygen/Gas Leakage Detection
RFID Module	Staff Authentication
Relay Module	Device Control
Buzzer	Emergency Alarm
LEDs	Status Indicators
OLED/LCD Display	Local Display
WiFi Router	Internet Connectivity
Laptop/PC	OpenCV Processing

Software Requirements

Software	Purpose
Arduino IDE	ESP32 Programming
Python	Computer Vision
OpenCV	Face Detection
Firebase	Cloud Database
ThingSpeak	IoT Data Visualization
Node-RED	IoT Dashboard Automation
HTML/CSS/JavaScript	Web Dashboard

Sensors Used and Their Functions

Medical Monitoring Sensors

Sensor	Function
MAX30102	Pulse + SpO2 Monitoring
LM35	Body Temperature
PIR Sensor	Patient Movement Detection
Camera Module	Face Detection
RFID	Staff Authentication
MQ Sensor	Hazardous Gas Detection

Computer Vision Integration

Use of OpenCV

Computer Vision can be used for:

- Face Detection
- Face Recognition
- Patient Monitoring
- Unauthorized Person Detection
- Mask Detection
- Fall Detection
- ICU Entry Monitoring

Python Libraries Required

opencv-python mediapipe numpy face_recognition cvzone requests firebase-admin

Cloud Platform Selection

Option 1: Firebase

Advantages

- Real-time database
- Authentication support
- Easy dashboard integration
- Faster response
- Remote monitoring

Best For

- Smart hospital dashboards
- Real-time updates
- Mobile applications

Option 2: ThingSpeak

Advantages

- Easy IoT analytics
- Graph visualization
- MATLAB support
- Simple integration

Best For

- Sensor data monitoring
- Data analytics
- IoT experiments

Recommended Approach

Use BOTH Platforms

Platform	Use
Firebase	Real-time dashboard
ThingSpeak	Graphs & analytics

Web Dashboard Design

Dashboard Modules

Patient Monitoring Panel

- Heart Rate
- Temperature
- Oxygen Level
- Room Status

Security Monitoring

- Face Detection Status
- Authorized Staff Entry
- Camera Feed

Emergency Alerts

- Emergency Notifications

- Critical Patient Alerts

Device Control

- Fan Control
- ICU Light Control
- Alarm Control

Technologies Used in Dashboard

Technology	Purpose
HTML	Structure
CSS	Styling
JavaScript	Dynamic Functions
Firebase Realtime DB	Live Data
Node-RED	IoT Flow Control

Remote Monitoring System

How Remote Access Works?

1. ESP32 connects to WiFi.
2. Data uploaded to Firebase/ThingSpeak.
3. Doctors access dashboard remotely.
4. Mobile or browser used anywhere.
5. Real-time alerts sent instantly.

Node-RED Integration

Why Node-RED?

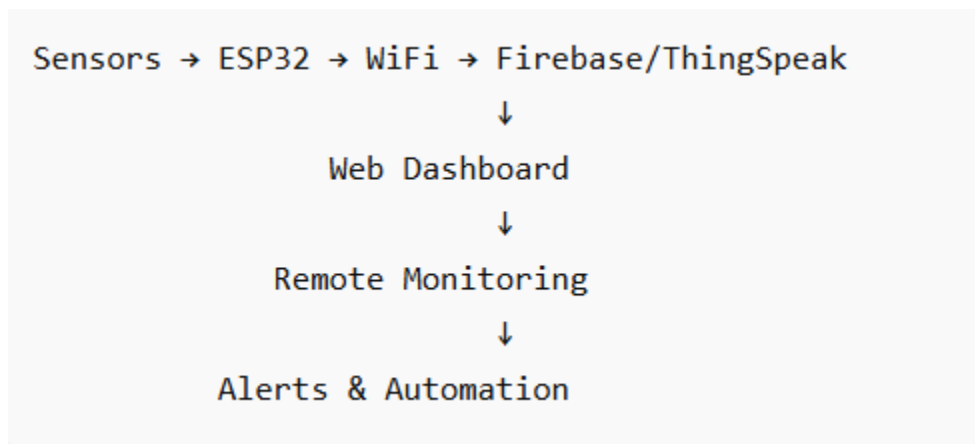
Node-RED provides:

- Drag-and-drop IoT flow
- Dashboard creation
- MQTT support
- Cloud connectivity

Node-RED Can:

- Receive sensor data
- Trigger alerts
- Store data
- Visualize hospital monitoring

Proposed System Architecture



Expected Results

The system should:

- Monitor patient data in real-time
- Detect faces successfully

- Upload data to cloud
- Display live dashboard updates
- Generate emergency alerts
- Support remote monitoring

Scope of the Case Study

Future Scope

AI Integration

- AI-based disease prediction
- Smart diagnosis systems

Advanced Computer Vision

- Fall detection
- Emotion detection
- Patient behavior analysis

Robotics

- Smart medicine delivery robots

Mobile App

- Android/iOS hospital monitoring app

Smart ICU

- Fully automated ICU management

Conclusion

The proposed IoT-Based Smart Hospital Management System using Computer Vision provides an intelligent healthcare solution by integrating IoT sensors, cloud computing, and AI technologies. The system improves hospital automation, patient monitoring, and security management while enabling real-time remote access and emergency handling.

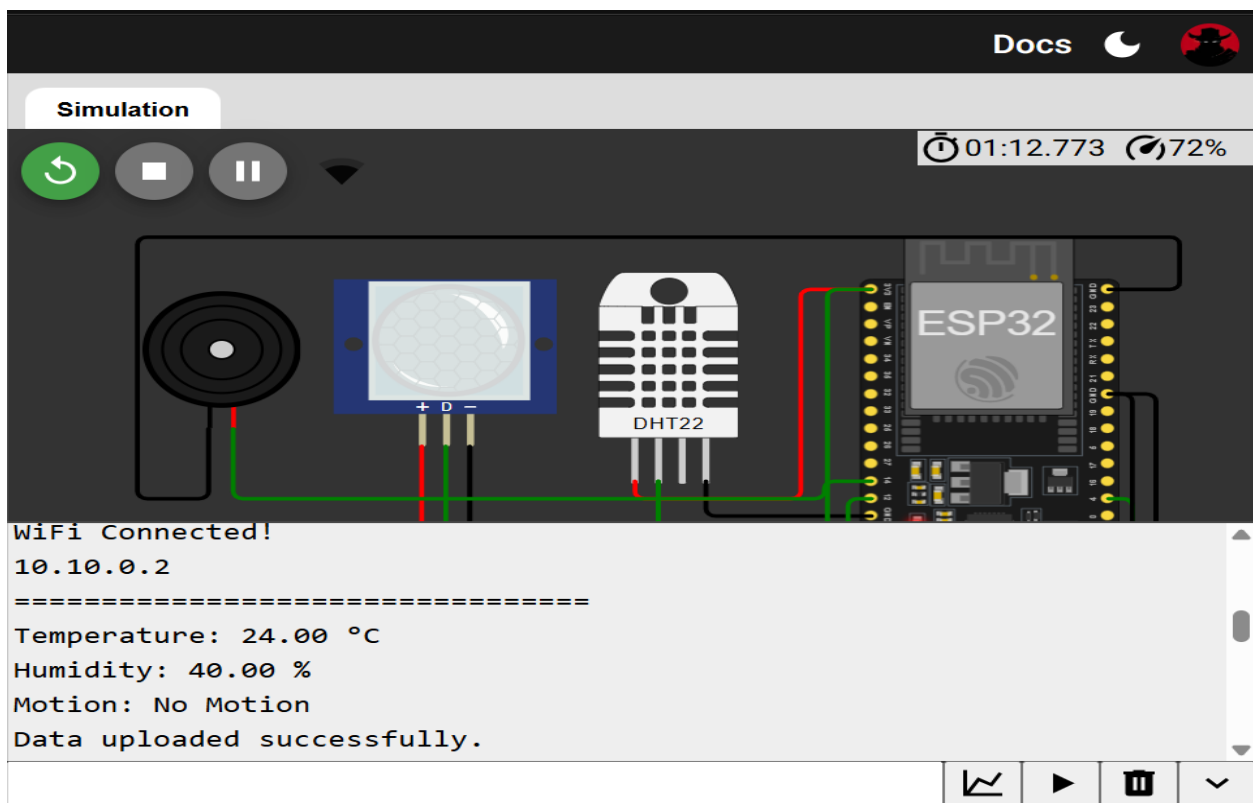
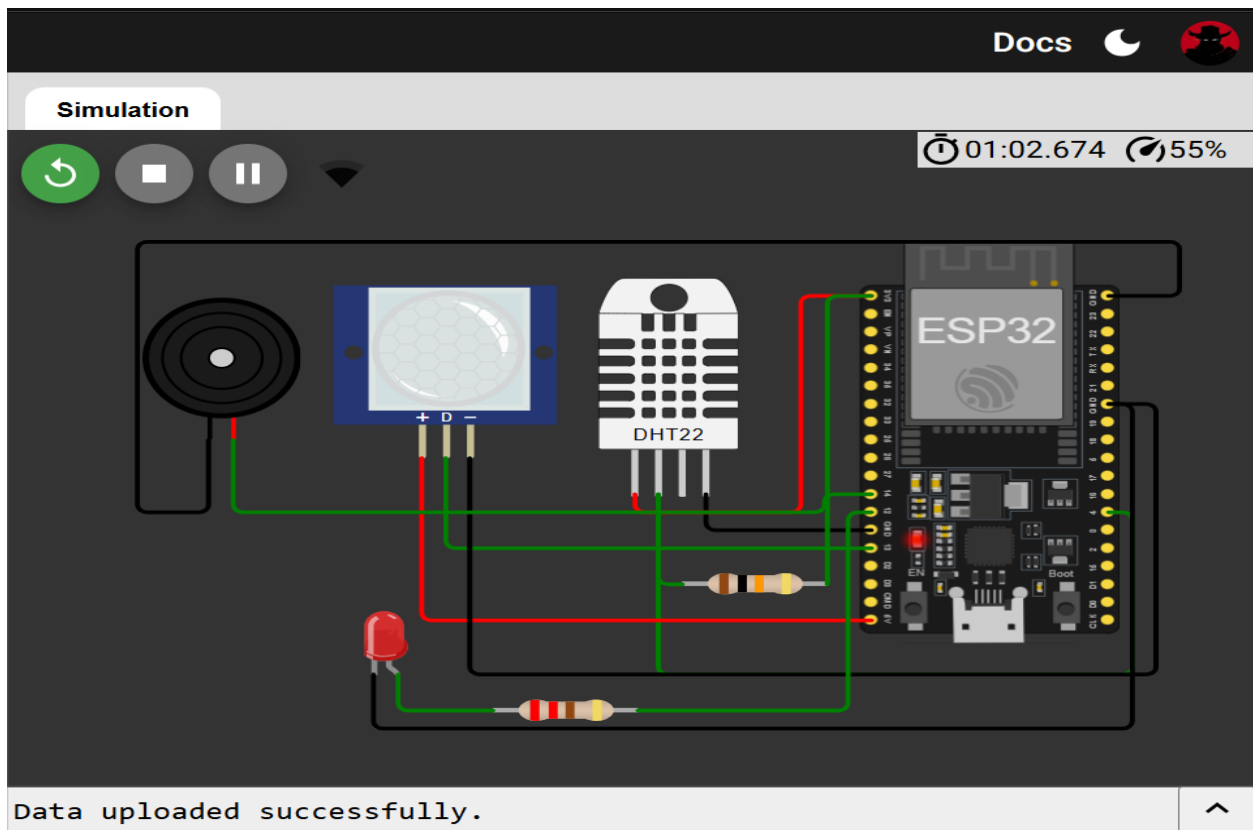
This case study is highly suitable for:

- IoT Course Projects
- Final Year Projects
- Computer Vision Labs
- Embedded System Research
- Smart Healthcare Research

LAB Review Questions

1. What is IoT and how is it used in smart healthcare systems?
2. Why is real-time monitoring important in hospitals?
3. What are the major problems in traditional hospital management systems?
4. Explain the role of Computer Vision in hospital automation.
5. What is the purpose of using ESP32 in this project?
6. Why are cloud platforms required in IoT healthcare systems?
7. Differentiate between Firebase and ThingSpeak.
8. What are the advantages of remote patient monitoring?
9. Explain the importance of automation in ICU monitoring systems.
10. What security challenges can occur in smart healthcare systems?
11. Which sensor is used for heart rate and oxygen monitoring?
12. Explain the working principle of the MAX30102 sensor.
13. Why is the PIR sensor used in hospital monitoring?
14. What is the role of RFID in this system?
15. Why is a buzzer used in emergency healthcare systems?
16. Explain how ESP32 communicates with cloud platforms.
17. What are the advantages of using WiFi-enabled microcontrollers?
18. How can gas sensors improve hospital safety?
19. Why are cameras important in Computer Vision-based healthcare systems?
20. Why is Firebase suitable for real-time dashboards?
21. Explain how ThingSpeak visualizes sensor data.
22. What are the advantages of cloud-based healthcare monitoring?
23. How can doctors access the system remotely?
24. What data can be stored in Firebase?
25. Explain the importance of real-time databases in IoT.
26. What is Node-RED and why is it useful?
27. How can notifications be generated in emergency conditions?
28. Explain the role of APIs in IoT dashboards.
29. Why is web dashboard security important?
30. How can AI-based disease prediction be added?
31. Explain the role of edge computing in healthcare IoT.
32. How can blockchain improve healthcare data security?
33. What are smart ICU systems?
34. Explain the use of digital twins in healthcare.
35. How can cloud AI services improve patient monitoring?

SOLUTION:





Smart Hospital Dashboard

Real-time IoT Monitoring using ESP32 + ThingSpeak

Channel ID
3301731

Temperature

24.0 °C

Humidity

40.0 %

Motion Status

No Motion

System Status

Online

Last update: 17/05/2026, 12:55:48

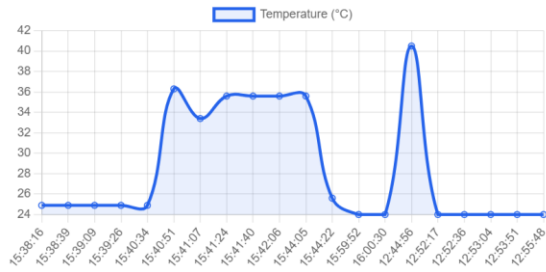
Temperature Trend



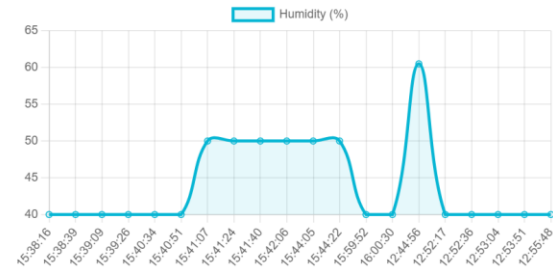
Humidity Trend



Temperature Trend



Humidity Trend



Recent Sensor Readings

Time	Temperature (°C)	Humidity (%)	Motion
17/05/2026, 12:55:48	24.0	40.0	No Motion
17/05/2026, 12:53:51	24.0	40.0	No Motion
17/05/2026, 12:53:04	24.0	40.0	Detected
17/05/2026, 12:52:36	24.0	40.0	No Motion
17/05/2026, 12:52:17	24.0	40.0	No Motion
17/05/2026, 12:44:56	40.5	60.5	No Motion
03/05/2026, 16:00:30	24.0	40.0	No Motion
03/05/2026, 15:59:52	24.0	40.0	No Motion
03/05/2026, 15:44:22	25.6	50.0	No Motion
03/05/2026, 15:44:05	35.6	50.0	No Motion
03/05/2026, 15:42:06	35.6	50.0	No Motion
03/05/2026, 15:41:40	35.6	50.0	No Motion

Increasing Temp & Humidity:

Docs

Simulation

01:38.816 46%

Editing DHT22

Temperature: 48.3°C

Humidity: 70.5%

Motion: No Motion

Data uploaded successfully.

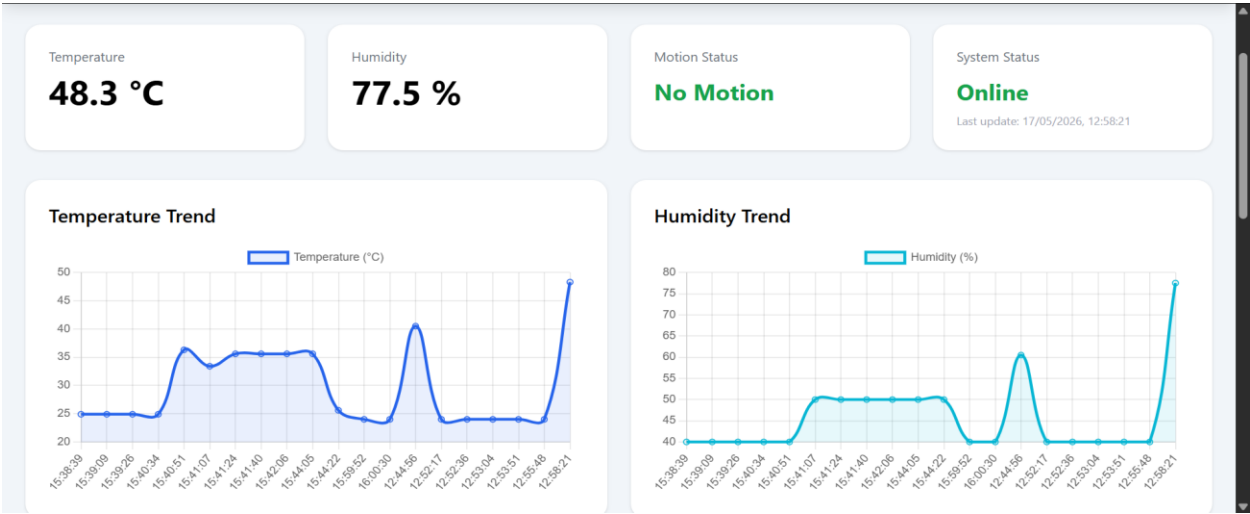
=====

Temperature: 48.30 °C

Humidity: 77.50 %

Motion: No Motion

Data uploaded successfully.



17/05/2026, 12:58:21	48.3	77.5	No Motion
17/05/2026, 12:55:48	24.0	40.0	No Motion
17/05/2026, 12:53:51	24.0	40.0	No Motion

Emergency Alert:

Simulation

02:01.591 94%

PIR Motion Sensor

Simulate motion

Motion: No Motion
Data uploaded successfully.
=====
Temperature: 48.30 °C
Humidity: 70.50 %
Motion: Detected
Data uploaded successfully.

Simulation controls: Play, Stop, Pause, Refresh, Wi-Fi, and a graph icon.

Simulation

02:07.774 40%

PIR Motion Sensor

Simulate motion

Data uploaded successfully.

Simulation controls: Play, Stop, Pause, Refresh, Wi-Fi, and a graph icon.



Smart Hospital Dashboard

Real-time IoT Monitoring using ESP32 + ThingSpeak

Channel ID

3301731

Temperature

48.3 °C

Humidity

70.5 %

Motion Status

Detected

System Status

Online

Last update: 17/05/2026, 13:00:33

🚨 Motion Detected! Emergency alert is active.

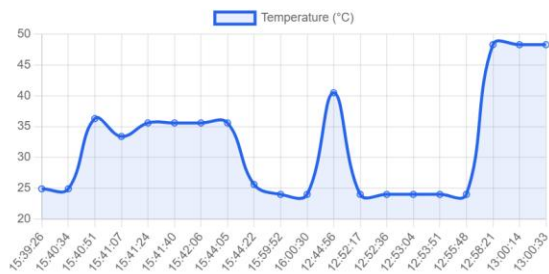
Temperature Trend



Humidity Trend



Temperature Trend



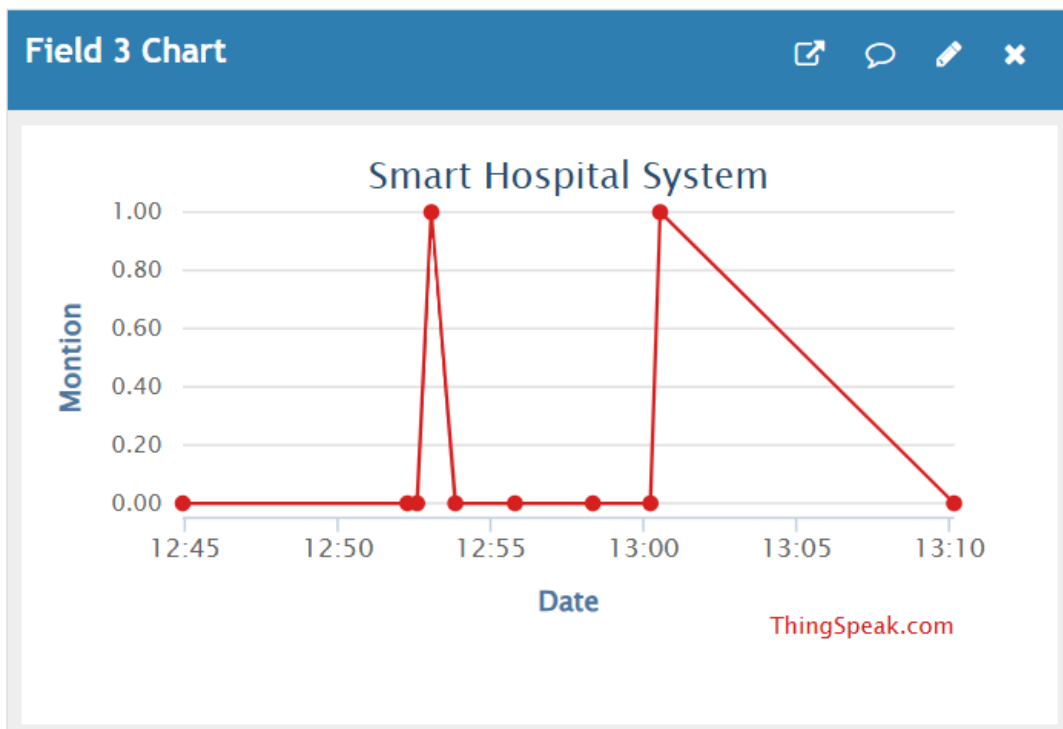
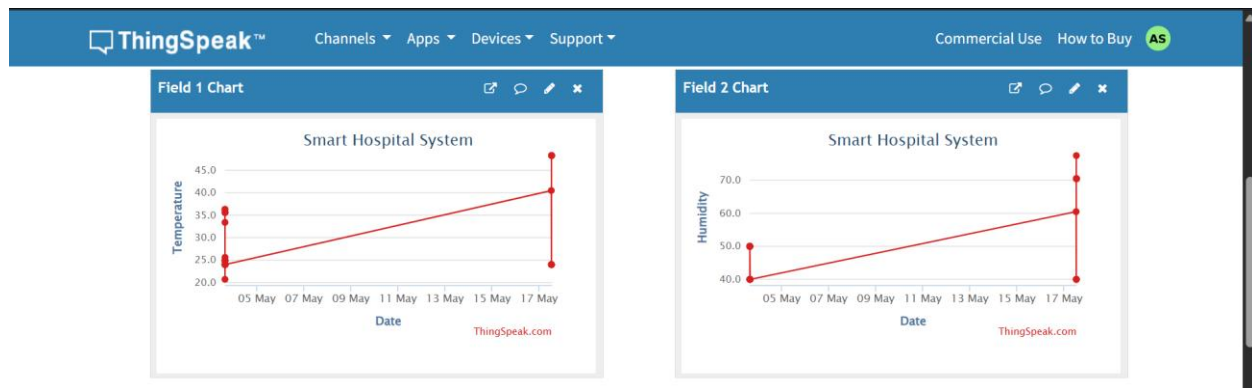
Humidity Trend



Recent Sensor Readings

Time	Temperature (°C)	Humidity (%)	Motion
17/05/2026, 13:00:33	48.3	70.5	Detected
17/05/2026, 13:00:14	48.3	70.5	No Motion
17/05/2026, 12:58:21	48.3	77.5	No Motion
17/05/2026, 12:55:48	24.0	40.0	No Motion
17/05/2026, 12:53:51	24.0	40.0	No Motion
17/05/2026, 12:53:04	24.0	40.0	Detected
17/05/2026, 12:52:36	24.0	40.0	No Motion
17/05/2026, 12:52:17	24.0	40.0	No Motion
17/05/2026, 12:44:56	40.5	60.5	No Motion
03/05/2026, 16:00:30	24.0	40.0	No Motion
03/05/2026, 15:59:52	24.0	40.0	No Motion
03/05/2026, 15:44:22	25.6	50.0	No Motion
03/05/2026, 15:44:05	25.6	50.0	No Motion

Thingspeak:



C++ code:

```
sketch.ino  diagram.json  libraries.txt  Library Manager  ▼

1  #include <WiFi.h>
2  #include <ThingSpeak.h>
3  #include <DHT.h>
4
5  // ===== WIFI =====
6  const char* ssid = "Wokwi-GUEST";
7  const char* password = "";
8
9  // ===== THINGSPEAK =====
10 unsigned long channelID = 3301731; // Your Channel ID
11 const char* writeAPIKey = "A3VKBHCTA5US6SVK"; // Replace with your API K
12
13 WiFiClient client;
14
15 // ===== PINS =====
16 #define DHTPIN 4
17 #define DHTTYPE DHT22
18
19 #define PIR_PIN 13
20 #define LED_PIN 12
21 #define BUZZER_PIN 14
22
23 // ===== OBJECTS =====
24 DHT dht(DHTPIN, DHTTYPE);
25
26 // ===== SETUP =====
27 void setup() {
28     Serial.begin(115200);
29
30     pinMode(PIR_PIN, INPUT);
31     pinMode(LED_PIN, OUTPUT);
32     pinMode(BUZZER_PIN, OUTPUT);
33
34     digitalWrite(LED_PIN, LOW);
35     digitalWrite(BUZZER_PIN, LOW);
36
37     dht.begin();
38
39     // Connect WiFi
40     Serial.print("Connecting to WiFi");
41     WiFi.begin(ssid, password);
42
43     while (WiFi.status() != WL_CONNECTED) {
44         delay(500);
45         Serial.print(".");
46     }
47
48     Serial.println("\nWiFi Connected!");
49     Serial.println(WiFi.localIP());
50 }
```

```

51 // Initialize ThingSpeak
52 ThingSpeak.begin(client);
53 }
54
55 // ===== LOOP =====
56 void loop() {
57 // Read DHT22
58 float temperature = dht.readTemperature();
59 float humidity = dht.readHumidity();
60
61 // Read PIR
62 int motion = digitalRead(PIR_PIN);
63
64 // Check DHT readings
65 if (isnan(temperature) || isnan(humidity)) {
66 Serial.println("Failed to read from DHT22!");
67 delay(2000);
68 return;
69 }
70
71 // Control LED and Buzzer
72 if (motion == HIGH) {
73 digitalWrite(LED_PIN, HIGH);
74 digitalWrite(BUZZER_PIN, HIGH);

```

```

75 } else {
76 digitalWrite(LED_PIN, LOW);
77 digitalWrite(BUZZER_PIN, LOW);
78 }
79
80 // Serial Monitor Output
81 Serial.println("=====");
82 Serial.print("Temperature: ");
83 Serial.print(temperature);
84 Serial.println(" °C");
85
86 Serial.print("Humidity: ");
87 Serial.print(humidity);
88 Serial.println(" %");
89
90 Serial.print("Motion: ");
91 Serial.println(motion ? "Detected" : "No Motion");
92
93 // Send data to ThingSpeak
94 ThingSpeak.setField(1, temperature);
95 ThingSpeak.setField(2, humidity);
96 ThingSpeak.setField(3, motion);
97

```

```

100 if (response == 200) {
101 Serial.println("Data uploaded successfully.");
102 } else {
103 Serial.print("ThingSpeak Error Code: ");
104 Serial.println(response);
105 }
106
107 // ThingSpeak requires at least 15 sec delay
108 delay(15000);
109 }

```